

FORGE FDE ACADEMY · DATA & KNOWLEDGE ARCHITECTURE

Data Discovery Semantic Layers · Knowledge Graphs Data Architecture Decisions

Every client asks for RAG. Most of them need something else — or something more. Today you learn to tell the difference, and to defend the answer with evidence rather than fashion.

Discovery

Semantic Layers

Knowledge Graphs

The Decision Tree

CONTINUITY

Where this session sits — and the failure it exists to prevent



Naive RAG

Hard chunking, top-K cosine. The failure — an entity mentioned once could not be linked to its description elsewhere.



Hybrid Retrieval

ADR-2 (locked): vector + keyword + RRF re-ranking. Better recall. Still passage-level — it finds text, not meaning.



This session

The decision: does this workload need metadata, a governed metric layer, a graph — or a hybrid of them? Defended with discovery evidence.



KG Studio

The build: Neo4j schema, advanced Cypher, entity resolution at scale, and the ADR-3 defence to your architect mentor.

The uncomfortable truth about ADR-2: hybrid retrieval fixed the SYMPTOM earlier demonstrated — poor recall on a single-mention entity. It did not fix the CAUSE: the relationships between entities were never made explicit anywhere. We got better at finding passages. We got no better at answering questions whose answer spans four documents and a version history.

What you will be able to decide by 17:00

Does the data even support this?

Discovery: exists · usable · governed · missing

Where should meaning live?

In text, metadata, metrics, or relationships

Which rung of the ladder?

Pure RAG → +metadata → +semantic → KG → hybrid

Can I defend it to a regulator?

Provenance is not the same thing as a citation

01



Data Discovery

What EXISTS · what is USABLE · what is GOVERNED · what is MISSING. Four questions, asked before any architecture is drawn — because the data decides the architecture, not the other way round.

DISCOVERY

Four questions — and the one everybody skips



1 · What EXISTS?

The inventory. Sources, formats, volumes, owners, refresh cadence. Read the pipelines and the storage bill, not the wiki.

Output: A source register with an owner per source.

Trap: the demo dataset. It is always clean, always small, and always unrepresentative.



2 · What is USABLE?

Profile it. Nulls, duplicates, mixed types, cardinality, encoding, dates stored as strings. Usable ≠ present.

Output: A profiling report per source, read by a human.

Trap: 'the data is clean' — nobody who says this has profiled it. Ask for the report, not the reassurance.



3 · What is GOVERNED?

Who owns it, who may see it, where may it live, how long is it kept, what is the authoritative source when two disagree?

Output: Classification + residency + system-of-record per entity.

Trap: assuming ungoverned means unregulated. It usually means regulated and unmanaged — which is worse.



4 · What is MISSING?

The question nobody asks. What must be true for the brief to work, that no source provides? Relationships. History. Ground truth.

Output: A gap register — each gap is a spike or a scope cut.

Trap: discovering the gap in UAT. It was findable on day one and it repriced the project.

HIGHLIGHT — question 4 is where the architecture is actually decided. “The relationship between a contract and the policy version in force when it was signed” exists in no source system. It is MISSING. You either build a layer that holds it — or you ship a system that guesses. There is no third option, and the regulator knows it.

DISCOVERY TOOLING

Profiling and expectations — discovery you can run, not discovery you assert

The loop: profile to DISCOVER what the data actually looks like → write expectations to ENFORCE what it must look like → gate the pipeline so the enforcement is not optional.



ydata-profiling — the discovery pass

- One command, one HTML report: distributions, nulls, duplicates, correlations, mixed types, high-cardinality warnings.
- Run it on the REAL extract on day one. The report is your evidence in the discovery conversation.
- Formerly Pandas Profiling — learners will meet both names in the wild.



Great Expectations — the enforcement pass

- Turn each profiling finding into a declarative, testable expectation.
- Run the suite in CI before every load. Fail = the load does not happen.
- Same GE + GitHub Actions gate from Session 1 — a new payload, not a new skill.

Findings → expectations (the discovery→gate handoff)

```
# profiling found: 3% null, 2 dupes, pct > 100
v.expect_column_values_to_not_be_null(
    column="counterparty_id" )
v.expect_column_values_to_be_unique(
    column="contract_id" )
v.expect_column_values_to_be_between(
    column="exposure_pct", min_value=0, max_value=100)
```



Why this gate sits BEFORE the architecture decision

A profiling report changes which architecture is even reachable.

If the counterparty name appears in four spellings across three sources, you do not have a retrieval problem — you have an entity-resolution problem, and no amount of embedding quality will fix it. The profile tells you that on day one, for the price of one command.

Discovery evidence is what turns “I think we need a graph” into “here is why RAG cannot answer this”.

The FDE move: never argue architecture from opinion. Run the profiler, put the report on the screen, and let the nulls make your case for you.

02



Schema Discovery Patterns

Schema-on-read · schema-on-write · contracts. Where you pay the tax for structure — and who pays it.

SCHEMA PATTERNS

Schema-on-write · schema-on-read · contracts — three answers to one question

The question every one of them answers: when do we discover that the data is the wrong shape — and who is standing there when we find out?



Schema-on-write

Structure enforced at load (write). Reject bad data at the door.

✓ Errors surface at write time, next to the producer who caused them. Readers can trust the shape.

✗ Rigid. Every source change is a migration. Slow to onboard new, messy sources.

Fits: Policy registers, ledgers, reference data — anything the business treats as authoritative.



Schema-on-read

Land raw; interpret at query (read) time. The reader imposes the shape.

✓ Fast to onboard anything. Nothing is lost — the raw artefact survives for re-interpretation.

✗ Errors surface at read time, in front of the consumer. Every reader re-implements the interpretation — and they disagree.

Fits: Unstructured corpora: contracts, emails, PDFs. You cannot schema a scanned side letter at the door.



Data contracts

An explicit, versioned agreement between producer and consumer, tested in CI.

✓ Moves the failure to the producer's pipeline BEFORE it reaches you. Turns an implicit assumption into a testable artefact with an owner.

✗ Needs a producer who will accept the obligation. Organisational, not technical, difficulty.

Fits: Every boundary you depend on and do not control. This is the grown-up answer.

The real-world answer is all three, by zone. Contracts land schema-on-read (raw PDFs, immutable, re-interpretable). The policy register is schema-on-write (authoritative, enforced). Between them sits a data contract, so the day the policy team adds a field, **their** CI fails — not your model's answer.

03



Semantic Layers

dbt · LookML · Cube · AtScale. One governed definition of every metric — and why this became an AI topic the moment we pointed a model at a warehouse.

SEMANTIC LAYERS · 1 OF 3

The problem a semantic layer solves — and why AI made it urgent



Semantic drift — the metric-definition epidemic

- Risk computes 'net exposure' one way. Finance computes it another. Both are in production. Both are 'right'.
- The definition lives in SQL scattered across dashboards, notebooks and one analyst's head.
- Nobody is wrong on purpose — the logic was re-derived per consumer because there was nowhere to put it once.
- The board meeting where two decks disagree is the symptom. The cause is architectural.



The semantic layer — define once, serve everywhere

- Metrics, dimensions, joins and access rules declared as code, version-controlled, reviewed.
- One definition served over APIs to dashboards, apps — and the LLM agent.
- The layer generates the SQL. Consumers select a metric; they do not re-invent it.
- Every query is attributable to a named, versioned definition — which is what an audit trail actually is.



WHY THIS IS AN AI TOPIC

Text-to-SQL against raw tables is a hallucination factory. The model guesses joins, invents grain, and re-derives 'revenue' differently on Tuesday than on Monday — and it does so *fluently*, which is worse than failing. Point the agent at a semantic layer instead: it selects a governed metric, the layer generates correct SQL. Accuracy stops being a property of your prompt and becomes a property of your architecture — which is the only kind that survives a model upgrade.



CASE STUDY — Airbnb's Minerva. Airbnb built a centralised metric platform after finding the same KPI computed differently across teams and dashboards — a pattern nearly every large enterprise reports independently. The engineering lesson is the ordering: they did not fix it by cleaning dashboards. They fixed it by moving the definition upstream of every consumer, so being consistent was easier than being inconsistent. **Britannia's version: Risk, Finance and the new AI assistant each 'compute' counterparty exposure. ADR-3 must name ONE owner of that definition — and the AI assistant must be a consumer of it, never an author.**

SEMANTIC LAYERS · 2 OF 3

dbt · LookML · Cube · AtScale — four tools, four centres of gravity



dbt (+ Semantic Layer / MetricFlow)

SQL transformation with tests, docs and lineage in version control; MetricFlow declares metrics once on top of the models.

Wins when — You already transform in dbt. The metric layer is adjacent to the models, reviewed in the same PR — lowest adoption friction of the four.



LookML (Looker)

Looker's modelling language: dimensions, measures and joins declared in code; Explores give governed self-service exploration.

Wins when — The organisation already runs Looker and its BI governance. Mature, battle-tested modelling — but its centre of gravity is the Looker ecosystem.



Cube

Headless BI: an open-source semantic layer serving the same definitions over REST, GraphQL and SQL APIs, with caching and row-level security.

Wins when — Many heterogeneous consumers — dashboards, apps, and an AI agent — need one governed surface, and you do not want a bundled dashboard product.



AtScale

Universal/OLAP-style semantic layer over the warehouse, with an MDX/BI heritage and aggregate-aware acceleration.

Wins when — Enterprise BI estates with heavy Excel/OLAP consumers and performance-sensitive aggregate workloads that must not be re-platformed.

Choose on gravity, not features. All four solve semantic drift competently. The question is never “which is best?” — it is “where does this organisation's metric logic already live, and which tool moves it the shortest distance?” The best semantic layer is the one the analytics team will actually maintain.

SEMANTIC LAYERS · 3 OF 3

Open Semantic Interchange — the standard that changes the buying decision

What it is: a vendor-neutral specification for expressing semantic models and metric definitions, so they become portable between tools instead of locked inside one. **It is not another semantic layer — it is a common format the existing ones can all speak.**

Sept 2025

OSI launched

Snowflake with Salesforce, dbt Labs, BlackRock, RelationalAI and a coalition of ecosystem partners.

Oct 2025

MetricFlow open-sourced

dbt Labs releases MetricFlow under Apache 2.0 as a reference implementation — a licence loosening, not a tightening.

Jan 2026

OSI v0.1 specification

The spec goes live; AtScale, Qlik and others join. Working groups cover metrics, composability and catalog integration.

Ongoing

Coalition keeps widening

Cube, Databricks, Oracle, Collibra, Informatica, ServiceNow, ThoughtSpot and many more listed as participants.

Why an FDE cares

- It reframes the question. “Which semantic layer do we buy?” becomes “does our semantic layer export a portable model?”
- It lowers the cost of being wrong — the strongest argument for adopting a layer at all.
- Agents are the driver: every vendor wants to be the governed context an AI reads. A shared format means you are not betting the metric estate on one of them.
- It is early. v0.1 is a specification, not a guarantee. Treat it as a direction of travel — and a question to ask a vendor.



The analogy that lands in the room: OSI is USB-C for semantics — not a new device, a common connector. And the honest caveat an FDE must add: a coalition announcement is not an implementation. Ask any vendor the same two questions — **“can you export your model to the OSI format today, and can you import someone else's?”** The answer separates the press release from the product. FDE’s should know how to ask it; not assume the answer.

04



Knowledge Graphs

Entities · relationships · ontologies. Neo4j · Amazon Neptune · Azure Cosmos DB (Gremlin). Making the connections explicit, so they can be queried instead of inferred.

KNOWLEDGE GRAPHS · 1 OF 3

Entities · relationships · ontologies — three words, three commitments



Entities

The things: Counterparty, Contract, Clause, Policy, PolicyVersion, Branch. One node per real-world thing — which is a promise you must keep with entity resolution.

Commitment: you will decide what counts as ONE thing. Three spellings of a company name is not three companies.



Relationships

First-class, typed, directed, with properties of their own: GOVERNED_BY {from, to}, AMENDS, REFERENCES, GUARANTEES. The edge carries meaning, not just connection.

Commitment: properties on edges are where time lives. 'Governed by, between these dates' is an edge property — and it is what makes a temporal answer possible.



Ontology

The agreed vocabulary and rules: what types exist, which relationships are legal between them, what a term means. The schema of meaning, not of storage.

Commitment: this is a business artefact, not a technical one. If Legal and Risk disagree on what 'in force' means, no database resolves it for you.

Hiring analogy: `(Candidate) -[INTERVIEWED_BY {date, round, verdict}]-> (Panelist)`.
The date belongs to the interview event — the edge — not to the candidate or the panelist.

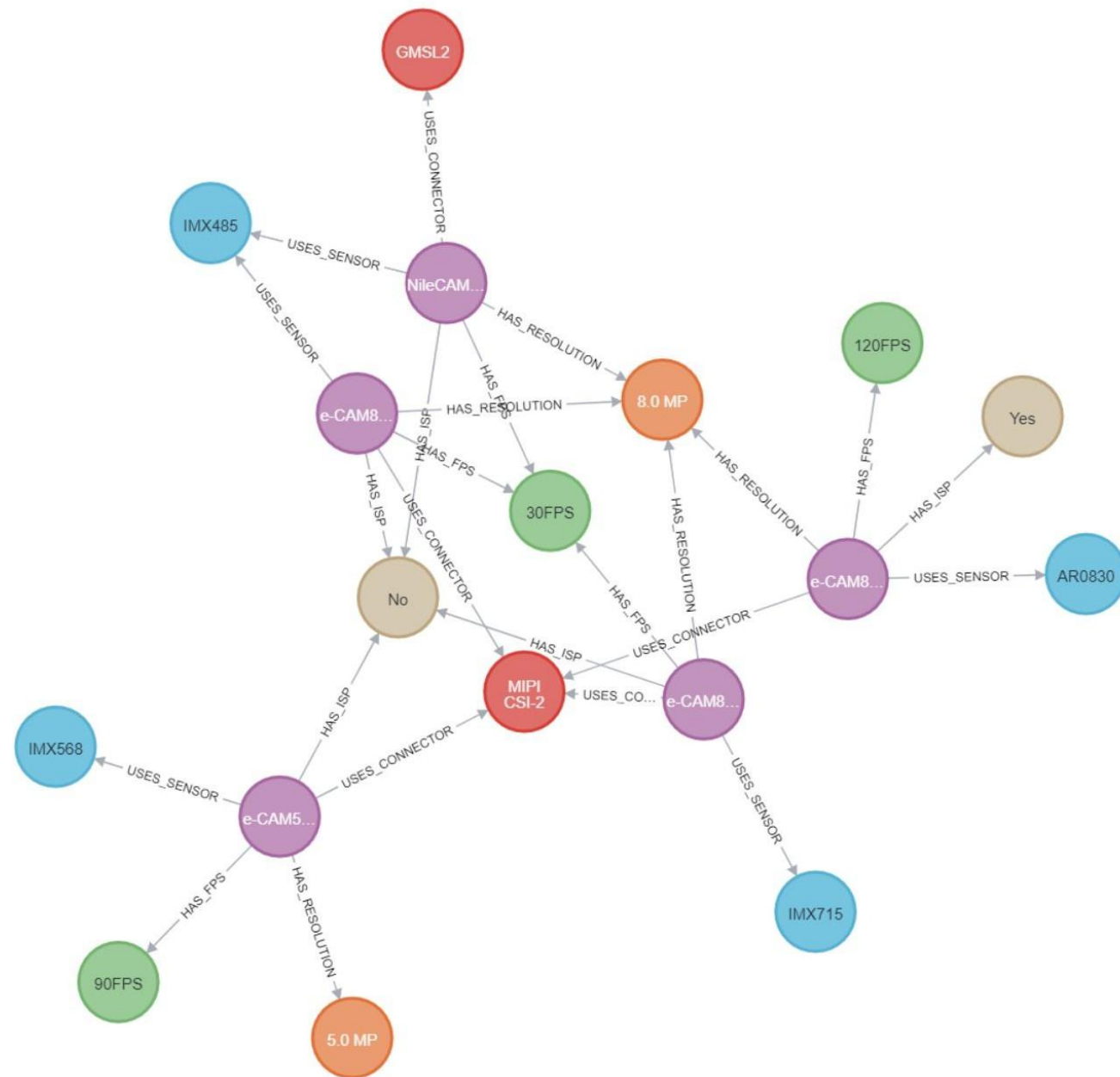
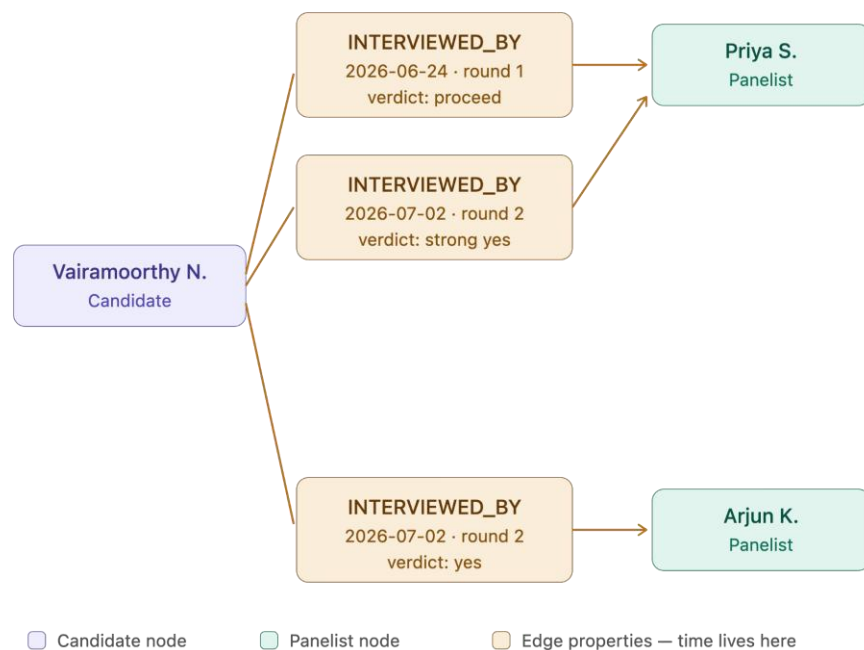


The ontology trap — and it is the most expensive mistake in this section. Teams spend six months building the Complete Enterprise Ontology and ship nothing. Ontologies are not modelled to completion; they are grown from questions. Model only what a named question requires — the ADR-3 rule of evidence: for every entity and relationship in your schema, name the question that needs it. If no question needs it, it does not go in the graph.

The engine is an ADR. The model is the asset. Entities, relationships and ontology survive a migration from Neo4j to Neptune. Your Cypher does not. Invest accordingly.

KNOWLEDGE GRAPHS · 2 OF 3

Graph - Examples



KNOWLEDGE GRAPHS · 3 OF 3

Neo4j · Amazon Neptune · Azure Cosmos DB (Gremlin) — pick on constraints, not on taste

Neo4j



Property graph. Cypher.

Strength — Richest graph ergonomics: Cypher fluency, mature tooling, the GDS algorithm library, first-class GraphRAG integrations.

Honest cost — A platform decision in its own right — another vendor, another support contract, another thing to run or buy as a managed service.

Amazon Neptune



Property graph (Gremlin + openCypher) AND RDF (SPARQL 1.1).

Strength — The only one of the three that speaks both property-graph and RDF models. Managed, multi-AZ. Neptune Analytics is a separate in-memory engine for global algorithms.

Honest cost — AWS-shaped. Neptune Analytics supports openCypher only and needs its own load — it is not simply 'analytics mode' on your database.

Azure Cosmos DB (Gremlin)



Property graph via the Apache TinkerPop Gremlin API.

Strength — Ticks Azure non-functionals by default: EU-West residency, SLAs, VNet integration, existing billing and governance. Graph is one model among several in one service.

Honest cost — Gremlin is imperative rather than declarative; no built-in GDS-style algorithm library. Ergonomics are the trade you make for procurement ease.

Query languages are not a detail — they are a portability decision. Cypher (Neo4j) and openCypher (Neptune) are close but have diverged; Gremlin (Cosmos, Neptune) is a different paradigm entirely — declarative pattern vs imperative traversal. Neptune's support for openCypher AND Gremlin on the same property graph is a genuine hedge. Cosmos's Gremlin-only surface is a genuine lock. Neither is wrong; both are ADRs with consequences.

05



When a Knowledge Graph Beats RAG

Multi-hop reasoning · entity resolution · provenance. Three query classes where better retrieval is not a smaller version of the right answer — it is the wrong shape entirely.

THE BOUNDARY

Three places RAG structurally cannot go



1 · Multi-hop reasoning

The answer exists in no chunk. It is computed by walking relationships across documents.

“Which lending exposures sit within two relationship hops of Meridian Freight?”

Why RAG fails: Retrieval returns the most similar passages about Meridian. None of them contains the exposure total — because that number has never been written down anywhere.



2 · Entity resolution

One real-world thing, many representations. Similarity treats them as different; the business needs them as one.

“Meridian Freight Ltd”, “MERIDIAN FREIGHT LIMITED”, “Meridian Frieght” — three sources, one company.

Why RAG fails: Embeddings may score these as similar, but 'similar' is not 'the same'. Sum an exposure across them and you either double-count or miss a third of it. Silently.



3 · Provenance

The regulator does not ask where the text came from. They ask why this answer is the answer.

“Show the clause, the policy version in force on the signature date, and the rule applied.”

Why RAG fails: A citation names a source. Provenance reconstructs a path. Cosine similarity produces neither — and it is not reproducible, which is the property an audit actually requires.

HIGHLIGHT — the distinction that wins the regulated deal: a citation is not provenance. A citation says *“this text came from document 47, page 12”*. Provenance says *“this answer holds because clause 8.2 of contract C-1188 invokes policy CP-114, the version in force on the signature date was v3, and v3’s threshold is 25%”* — a path, reproducible on demand, identical every time it is asked. Most vendors sell the first and call it the second.

06



When the Semantic Layer Beats Both

Governed metrics · audit trail · consistent definitions. The layer nobody demos, that quietly decides whether the numbers can be trusted.

THE OTHER BOUNDARY

Where a graph is the wrong tool and RAG is a liability

The tell: the question has ONE correct answer, computed by an agreed rule, over structured data — and the organisation must be able to prove it computed that rule and no other.



Governed metrics

“What is our net exposure to this counterparty?” is not a retrieval question and not a traversal question. It is arithmetic over a definition — and the definition is a governance decision, not a modelling one.

vs the alternatives: A graph would traverse to the right nodes and then still have to compute the metric. Why would that arithmetic be re-implemented in Cypher when Finance already owns it in the metric layer?



Audit trail

Every query resolves to a named, versioned metric definition, in version control, with a reviewer and a commit. The audit answer already exists as a git history.

vs the alternatives: RAG's answer to 'why this number?' is a similarity score. The semantic layer's answer is a pull request with a name on it. Only one of those is admissible.



Consistent definitions

The AI assistant, the risk dashboard and the regulatory return must produce the SAME number for the same question — on the same day and in six months.

vs the alternatives: Consistency cannot be prompted into a model. It has to be structurally impossible to be inconsistent — which means one definition, one generator, many consumers.

The sentence to carry into ADR-3: RAG answers “what does the document say?” · a knowledge graph answers “how are these things connected?” · a semantic layer answers “what is the agreed number?”. Three different questions. A production system routes each one to the layer that owns it — and the architecture's quality is the quality of that routing decision, not the sophistication of any single layer.

07



The Data Architecture Decision Tree

Pure RAG → RAG + metadata → RAG + semantic layer → knowledge graph → hybrid. A ladder you climb only when the evidence forces you up a rung.

THE LADDER

Five rungs — climb only when the evidence forces you

The governing rule: start at rung 1 and climb only when a NAMED question fails. Every rung adds capability and cost — cost you must justify, staff, and keep alive after you leave.

1	Pure RAG	<i>Chunk, embed, retrieve, generate.</i>	Unstructured corpus · questions answerable from a single passage · no arithmetic · no audit burden.	↑ Climb when: retrieval returns the right document but the wrong section, or you cannot filter by date/type/owner.
2	RAG + metadata	<i>Add structured fields to every chunk: doc type, date, owner, version, jurisdiction. Filter before you rank.</i>	Cheapest real win available. Most 'we need a graph' conversations end here, honestly.	↑ Climb when: the question needs a NUMBER that must match the official one, or the answer needs a governed definition.
3	RAG + semantic layer	<i>Route metric questions to governed definitions; route text questions to retrieval.</i>	Mixed estates: unstructured text plus authoritative structured data with agreed metrics and an audit requirement.	↑ Climb when: answers require traversing relationships across documents, or one thing has many identities.
4	Knowledge graph	<i>Entities and relationships made explicit; multi-hop traversal; entity resolution; provenance paths.</i>	Relationship-shaped questions · fraud rings · ownership chains · dependency maps · derivation paths.	↑ Climb when: you ALSO have a large unstructured corpus and governed metrics — i.e. almost every real enterprise.
5	Hybrid + router	<i>All three layers behind a router that sends each question class to the layer that owns it.</i>	The production answer for mixed estates. Also the most expensive thing to operate and the easiest to over-build.	↑ Do NOT start here. Arrive here with evidence, one rung at a time, or you will build three systems and govern none.

Rung 2 is the honest default and the most-skipped rung. Metadata filtering is unglamorous, costs days not quarters, and resolves a surprising share of “we need a knowledge graph” briefs. Try it before you spend the client's money on rung 4.

DECISION AID

The router — sort the question, not the technology

How to use it: take ten real questions from the client's own mouth. Sort each into a row. The distribution IS your architecture — and it is evidence, not opinion.

“What does the policy say about X?”	Answer lives in one passage.	Pure RAG
“What did the policy say in March 2021?”	Versioned + temporal filter, still one passage.	RAG + metadata
“What is our net exposure to this counterparty?”	One agreed number, arithmetic over structured data, must be auditable.	Semantic layer
“Which facilities are governed by CP-114 v3?”	Cross-document reference resolution across a version history.	Knowledge graph
“Is this contract compliant, and prove it?”	Clause text + policy version in force + threshold + derivation path.	Hybrid + router

Failure mode A — under-building: every question routed to RAG because RAG is what was funded. The last two rows return confident, wrong answers.

FORGE FDE Academy · Data Discovery, Semantic Layers & Knowledge Graphs | Confidential · Trainer Use Only

Failure mode B — over-building: a graph built for row 4 when nine of ten real questions were row 1. Correct architecture, wrong question distribution.

08



Case Study — “We Want RAG”

70% unstructured contracts referencing 30% structured policy data. The regulator wants every answer traceable. Choose your architecture — and defend it.

THE BRIEF

Britannia Counties Bank — contract compliance, and the word nobody defined

THE BRIEF: “We want RAG over our lending contracts, so anyone can ask whether a facility complies with credit policy. The regulator wants every answer traceable.”

The estate — what discovery found



70% — unstructured contracts

- Master agreements, facility letters, amendments, side letters, covenant schedules. PDFs, some scanned.
- Contracts reference policies by ID: “subject to the Bank's credit policy CP-114 as amended from time to time”.
- An amendment chain: C-1188 amended twice; the side letter varies a covenant but not the policy reference.



30% — structured policy data

- Credit policy register: policy_id, version, effective_from, effective_to, thresholds, approver.
- Counterparty master and limits — the authoritative source, schema-on-write, well governed.
- Profiling found counterparty names in four spellings across three source systems.



The question that decides the architecture — ask it in minute one: “Does facility C-1188 comply with credit policy CP-114?” To answer, you must know: which clause governs · which policy version was in force **on the signature date** · what that version's threshold was · and whether an amendment changed either. That answer exists in no document, no chunk, and no single table.

Why the client's instinct is wrong — but reasonable

“CP-114” matches everywhere — The string appears in 400 contracts across 6 policy versions. Similarity cannot pick the right one — they are all identical text.

“As amended from time to time” — The contract deliberately does NOT name a version. The applicable version is computed from dates, not retrieved from text.

Traceable ≠ cited — The regulator wants a reproducible derivation, identical in six months. Re-embed the index and RAG's answer may move.

THE DECISION

Walking the ladder — and the named question that fails at each rung

1	Pure RAG	<i>“What did CP-114 require on 4 March 2021?”</i>	Six versions, near-identical text. Retrieval returns the most similar — which is effectively arbitrary among them. Confidently wrong.
2	RAG + metadata	<i>Same question, now filterable by effective_from / effective_to.</i>	Genuine progress — this now answers the policy question correctly. But it cannot resolve WHICH version applies to C-1188, because that needs the contract's signature date joined to the version history.
3	+ Semantic layer	<i>“What is the exposure, and does it breach the threshold?”</i>	Now the arithmetic is governed: one definition of exposure, one threshold lookup, version-controlled, auditable. Handles the 30% properly. Still cannot walk the amendment chain.
4	Knowledge graph	<i>“Which clause governs, after two amendments and a side letter?”</i>	Contract → AMENDED_BY → Amendment → VARIES → Clause → REFERENCES → Policy → HAS_VERSION → v3 {effective_from, effective_to}. Multi-hop, temporal, and entity-resolved. This is the answer.
5	Hybrid + router	<i>All of the above, from one question box.</i>	Route by question class. Text → hybrid retrieval (ADR-2 stands). Number → semantic layer. Relationship/derivation → graph. Compliance question → all three, assembled with a provenance path.

Note what this is: not “we chose hybrid because it is the most complete”. It is four rejected rungs, each with the named question that killed it. That is the difference between an architecture and a preference.

THE ANSWER

The architecture — and how an answer is assembled

Question router

Classifies the question: text · number · relationship · compliance. Logged with the answer — the routing decision is itself auditable.

Text layer — hybrid retrieval

ADR-2 stands: vector + keyword + RRF over clause-level chunks, with metadata (doc type, date, counterparty, policy refs).

Metric layer — semantic layer

Governed definitions: exposure, threshold, breach. One definition, version-controlled, served to dashboard and agent alike.

Graph layer — knowledge graph

Contract · Amendment · Clause · Policy · PolicyVersion · Counterparty. Temporal validity on edges. Entity resolution on counterparties.

Provenance assembler

Every answer returns the path: clause id → policy id → version → effective window → threshold → rule applied → verdict.

The 70/30 misread. The client saw 70% unstructured and concluded “this is a RAG problem”. But the 30% is not a minority partner — it is the **authority**. The contracts ask the questions; the policy register answers them. Volume is not importance, and the split that matters is not 70/30 by size — it is which side carries the truth.

The answer the regulator accepts

```
VERDICT: compliant
# because — the derivation path:
clause      C-1188 §8.2
amended     A-2  (2022-11-04)
refs        CP-114
signed      2021-03-04
version     v3 (in force
            2020-06-01 → 2023-01-31)
threshold   25%  measured 18.4%
```



Why this survives an audit

- Reproducible: re-run in six months, same path, same verdict — no embedding drift in the reasoning.
- Attributable: the threshold came from a named, version-controlled definition with a reviewer.
- Inspectable: every hop is a fact a human can challenge individually.

THE DEFENCE

The attack bank — what your architect mentor will actually ask

“This is over-engineered. Why not just RAG with good metadata?”

Rung 2 answers the policy question — I showed that, and I kept it. It cannot join a contract's signature date to a version history across an amendment chain. Here is the question it fails: C-1188, after two amendments. Give me a metadata schema that answers it and I will drop the graph.

“You're building three systems. Who operates them at 3am?”

Fair, and it is the strongest objection. My answer: the graph holds only what a named question requires — six node types, not an enterprise ontology. The semantic layer is dbt-adjacent, owned by the analytics team that already exists. I added one new runtime, not three.

“The LLM could just read the policy register. Why a semantic layer?”

It could, and it would compute exposure a slightly different way each time — fluently. The regulator does not want a good number; they want the SAME number as the regulatory return. That is a structural guarantee, not a prompt.

“What if the ontology is wrong?”

Then a named question returns a wrong path, and the path is inspectable — which is how we find out. That is the point of provenance: it makes our errors visible rather than plausible. A RAG error looks like an answer.

“Why not Neptune or Cosmos instead of Neo4j?”

Both viable — the model ports, the Cypher does not. Cosmos wins if procurement and EU-West residency dominate; Neptune wins if we need RDF alongside. That is an engine ADR with a stated trigger to revisit, not a religion.

“Prove the graph earns its cost.”

Ten real questions from the client. Eight need the graph or the metric layer. If it had been two of ten, I would have shipped rung 2 and told you so

Golden rule of the defence: never defend what you cannot evidence. “We measured X; we assumed Y; we would revisit if Z” beats any adjective — and “I would have shipped the smaller thing” is the most credible sentence in the room.

WRAP-UP

What you shipped today — and what Week 3 does next



Shipped this session

- Discovery in four questions — exists · usable · governed · missing — with profiling as the evidence, not opinion.
- Schema patterns as a tax question: who pays for structure, and when.
- Semantic layers: what they solve, four tools by centre of gravity, and OSI as the emerging portable format.
- Knowledge graphs: entities, relationships, ontologies — and the trap of modelling to completion.
- The three boundaries: multi-hop, entity resolution and provenance for the graph; governed metrics and audit for the semantic layer.
- The five-rung ladder, and the rule that you climb only when a named question fails.



Next session — Knowledge Graph Studio

- You decided the architecture today. Next you BUILD it: Neo4j schema, advanced Cypher, multi-hop path queries.
- Entity resolution at scale — the four spellings of Meridian Freight become one node, reversibly, with provenance.
- Index design and query optimisation: read the plan, not the vibes.
- ADR-3 defended to your architect mentor in ten minutes, using this attack bank.
- Bring: your rung choice, and the named question that killed the rung below it.

The through-line: the client asked for RAG because RAG is what they had heard of. They were not wrong about the data — they were wrong about where meaning lives in it. Seventy per cent of the bytes were contracts; one hundred per cent of the **authority** was in the register, and all of the **reasoning** was in the relationships between them. Your job is not to give clients the architecture they asked for, nor the most impressive one you can build. It is to find where meaning lives — and put the machinery there.

Next: Knowledge Graph Studio

build it, then defend it.

Bring your rung choice and the named question that killed the rung below. We build the Britannia graph in Cypher, resolve the entities, and you defend ADR-3 to an architect who has read this attack bank.

Neo4j · Cypher

Entity resolution

ADR-3 defence